TemporalDifference

Implementazione "from scratch" e analisi sperimentale
di algoritmi di Reinforcement Learning su CartPole, Acrobot e Pong

Federico Sabbadini
M.Sc. Computer Engineering — Università degli Studi di Brescia
Corso di Reinforcement Learning, A.A. 2025/26

10 luglio 2026

Sommario

Questo progetto implementa da zero i principali algoritmi tabellari di Reinforcement Learning (Q-Learning, SARSA, Monte Carlo e TD(0)) e un agente Deep Q-Network (DQN) in PyTorch, applicandoli a tre ambienti Gymnasium: CartPole, Acrobot e Pong. L'obiettivo, in linea con la consegna, non è ottenere performance elevate ma *comprendere* il comportamento degli algoritmi e analizzarne criticamente i risultati. Tutti gli esperimenti sono mediati su 3 seed con bande di deviazione standard. Il lavoro include analisi di sensibilità agli iperparametri, casi di fallimento espliciti, il confronto SARSA vs Q-Learning e MC vs TD(0), e uno studio di ablazione del DQN (target network ed experience replay) con il bonus Double DQN. L'analisi mette in evidenza anche i risultati che *contraddicono* le ipotesi iniziali e i limiti metodologici individuati.

Indice

1 Introduzione **2**
1.1 Obiettivi del progetto . . . 2
1.2 Ambienti . . . 3
1.3 Algoritmi . . . 3
1.4 Struttura del progetto e del report . . . 3

2 Fondamenti teorici **3**
2.1 MDP, valore e policy . . . 3
2.2 Predizione vs controllo . . . 3
2.3 Monte Carlo (MC) e TD(0) . . . 4
2.4 SARSA (on-policy) e Q-Learning (off-policy) . . . 4
2.5 Esplorazione: ε-greedy . . . 4
2.6 Discretizzazione (state aggregation) . . . 4
2.7 Deep Q-Network (DQN) . . . 4

3 Metodologia e implementazione **4**
3.1 Organizzazione del codice . . . 4
3.2 Ambienti: stati, azioni, reward . . . 5
3.3 Policy di predizione valutata . . . 5
3.4 Architettura del DQN . . . 5
3.5 Training vs evaluation . . . 5
3.6 Riproducibilità . . . 5

4 Setup sperimentale **6**
4.1 Hardware e librerie . . . 6
4.2 Iperparametri . . . 6
4.3 Esperimenti eseguiti . . . 6
4.4 Criteri di valutazione . . . 6

5 Risultati **7**
5.1 CartPole — controllo . . . 7
5.2 CartPole — sensibilità agli iperparametri . . . 8
5.3 CartPole — casi di fallimento . . . 8
5.4 CartPole — MC vs TD(0) (predizione) . . . 8
5.5 Acrobot — controllo . . . 9
5.6 Acrobot — risoluzione della discretizzazione . . . 10
5.7 Acrobot — sensibilità a ε e predizione . . . 11
5.8 Pong — studio di ablazione del DQN . . . 12

6 Analisi critica **13**
6.1 L'agente sta davvero imparando? . . . 13
6.2 Convergenza, stabilità e varianza . . . 13
6.3 Riproducibilità . . . 14
6.4 Training vs evaluation, checkpoint . . . 14
6.5 Limiti metodologici individuati . . . 14
6.6 Buono per un progetto didattico, non "research-grade" . . . 15

7 Analisi sperimentale e lezioni apprese **15**
7.1 Obiettivi iniziali di ciascun esperimento . . . 15
7.2 Cosa ha funzionato . . . 15
7.3 Cosa non ha funzionato (o ha sorpreso) . . . 15
7.4 Problemi incontrati durante lo sviluppo . . . 16
7.5 Correzioni applicate . . . 16
7.6 Limiti residui . . . 17
7.7 Possibili miglioramenti futuri . . . 17

8 Conformità alla consegna **17**

9 Origine del codice **18**

10 Conclusioni **19**
10.1 Risultati raggiunti . . . 19
10.2 Cosa si è imparato . . . 19
10.3 Possibili miglioramenti futuri . . . 20
10.4 Valutazione complessiva . . . 20

---

1 Introduzione

1.1 Obiettivi del progetto

Il progetto ha lo scopo di implementare *da zero* i principali algoritmi di Reinforcement Learning (RL) e di confrontarli su tre ambienti di difficoltà crescente, analizzandone comportamento, limiti e prestazioni. Coerentemente con la consegna, l'enfasi è sulla **comprensione** e sull'**analisi**

sperimentale critica, non sulla performance assoluta: molti risultati sono volutamente sub-ottimali e vengono analizzati come tali, inclusi i casi di fallimento e i risultati che smentiscono le ipotesi di partenza.

1.2 Ambienti

Sono stati affrontati tre ambienti Gymnasium [4]:

- **CartPole-v1** (comune a tutti i gruppi): bilanciare un'asta su un carrello. Stato continuo 4-D, 2 azioni, reward $+1$ per passo.
- **Acrobot-v1** (assegnato): portare la punta di un doppio pendolo sopra una soglia. Stato continuo 6-D, 3 azioni, reward -1 per passo (reward sparso).
- **ALE/Pong-v5** (assegnato): il gioco Atari Pong, affrontato con Deep RL su osservazioni a pixel [1].

1.3 Algoritmi

Per CartPole e Acrobot si utilizzano quattro algoritmi tabellari implementati a mano: due di *controllo* (Q-Learning, SARSA) e due di *predizione* (Monte Carlo, TD(0)). Poiché gli ambienti hanno stato continuo, è stato necessario introdurre una *discretizzazione* dello spazio degli stati. Per Pong si è implementato un DQN in PyTorch [2] con experience replay e target network, più il bonus Double DQN [5].

1.4 Struttura del progetto e del report

Il codice è organizzato nel pacchetto `rl_lab` (algoritmi, ambienti, utility) con uno script per ambiente in `scripts/`. Il report segue il percorso: fondamenti teorici (Sez. 2), metodologia (Sez. 3), setup sperimentale (Sez. 4), risultati (Sez. 5), analisi critica (Sez. 6), conformità alla consegna (Sez. 8), origine del codice (Sez. 9) e conclusioni (Sez. 10).

2 Fondamenti teorici

Si riportano solo i concetti necessari a comprendere le implementazioni. La notazione segue Sutton & Barto [3].

2.1 MDP, valore e policy

Un problema di RL è modellato come un *Markov Decision Process* (MDP): ad ogni *timestep* l'agente osserva uno *stato* s, sceglie un'*azione* a secondo una *policy* $\pi(a|s)$, riceve un *reward* r e transita in s'. L'obiettivo è massimizzare il *return* atteso $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, dove $\gamma \in [0, 1]$ è il fattore di sconto. Si definiscono la *state-value* $V^\pi(s) = \mathbb{E}_\pi[G_t \,|\, S_t = s]$ e la *action-value* $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \,|\, S_t = s, A_t = a]$.

2.2 Predizione vs controllo

È fondamentale distinguere i due compiti, perché la consegna li richiede entrambi:

- **Predizione**: stimare V^π per una policy π *fissata*. Monte Carlo e TD(0) sono algoritmi di predizione.
- **Controllo**: trovare una policy migliore. Q-Learning e SARSA sono algoritmi di controllo.

2.3 Monte Carlo (MC) e TD(0)

Entrambi stimano V^π ma con target diversi (S&B §5.1, §6.1):

$$\text{MC:} \quad V(S_t) \leftarrow V(S_t) + \alpha\,[\,G_t - V(S_t)\,] \tag{1}$$

$$\text{TD(0):} \quad V(S_t) \leftarrow V(S_t) + \alpha\,[\,R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\,] \tag{2}$$

MC usa il return *effettivo* G_t: è non distorto (*unbiased*) ma ad alta varianza e richiede episodi completi. TD(0) usa il *bootstrap* $R_{t+1} + \gamma V(S_{t+1})$: è distorto (usa una stima) ma a varianza più bassa e aggiornabile online. Il termine $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ è il *TD error*.

2.4 SARSA (on-policy) e Q-Learning (off-policy)

Sono aggiornamenti TD sulla Q (S&B §6.4–6.5):

$$\text{SARSA:} \quad Q(S,A) \leftarrow Q(S,A) + \alpha\,[\,R + \gamma Q(S',A') - Q(S,A)\,] \tag{3}$$

$$\text{Q-Learning:} \quad Q(S,A) \leftarrow Q(S,A) + \alpha\,[\,R + \gamma \max_{a'} Q(S',a') - Q(S,A)\,] \tag{4}$$

SARSA è *on-policy*: il target usa l'azione A' effettivamente scelta dalla policy ε-greedy, quindi tiene conto del rischio dell'esplorazione. Q-Learning è *off-policy*: il target usa il max, cioè valuta la policy greedy indipendentemente da come si esplora.

2.5 Esplorazione: ε-greedy

La policy ε-greedy sceglie un'azione casuale con probabilità ε (*exploration*) e l'azione greedy $\arg\max_a Q(s,a)$ altrimenti (*exploitation*). È il compromesso più semplice tra i due e viene usato in tutto il progetto (S&B §2.1).

2.6 Discretizzazione (state aggregation)

I metodi tabellari richiedono uno spazio di stati finito. CartPole e Acrobot hanno stato continuo, quindi ogni dimensione viene divisa in *bin* uniformi (S&B §9.4, *state aggregation*): stati vicini finiscono nello stesso bin. Il numero di stati cresce come $(\text{bin})^{\dim}$: 10^4 per CartPole (gestibile), fino a 10^6 per Acrobot con 10 bin (*curse of dimensionality*).

2.7 Deep Q-Network (DQN)

Quando lo spazio degli stati è troppo grande (pixel di Pong), la Q viene approssimata da una rete neurale $Q(s,a;\theta)$ [2]. Due componenti stabilizzano l'addestramento:

- **Experience Replay**: le transizioni sono salvate in un buffer e campionate a mini-batch casuali, rompendo la correlazione temporale.
- **Target Network**: una copia θ^- della rete, aggiornata di rado, fornisce target stabili $y = R + \gamma \max_{a'} Q(S',a';\theta^-)$.

Il **Double DQN** [5] disaccoppia selezione e valutazione dell'azione, $y = R + \gamma\, Q(S', \arg\max_{a'} Q(S',a';\theta);\theta^-)$, riducendo la sovrastima (*maximization bias*, S&B §6.7).

3 Metodologia e implementazione

3.1 Organizzazione del codice

Il codice è nel pacchetto `rl_lab`, con separazione netta delle responsabilità:

- `algorithms/tabular/`: Q-Learning, SARSA, MC, TD(0) — funzioni pure (ambiente in ingresso, risultati in uscita).

- `algorithms/deep/`: rete (`networks.py`), replay buffer, loop di training DQN con flag di ablazione e Double DQN.
- `envs/`: `discretizer.py` (binning generico) e `atari_wrappers.py` (preprocessing Atari).
- `experiments/runner.py`: esecuzione multi-seed.
- `utils/`: seeding e plotting (curve con banda $\pm 1\sigma$).
- `scripts/`: uno script per ambiente che orchestra gli esperimenti.

Gli iperparametri di ambiente stanno in `configs.py`; il plotting è isolato dai loop di training.

3.2 Ambienti: stati, azioni, reward

Tabella 1: Caratteristiche degli ambienti e discretizzazione adottata.

Ambiente	Stato	Azioni	Reward	Discretizzazione
CartPole	4-D cont.	2	$+1$/passo	10 bin/dim $\rightarrow 10^4$ stati
Acrobot	6-D cont.	3	-1/passo	6 bin/dim $\rightarrow$ 46656 stati
Pong	210×160 RGB	6	± 1 (punto)	— (pixel, DQN)

Discretizzazione. Il wrapper `DiscretizedEnv` clippa ogni dimensione in un intervallo configurato, la scala su $[0, \text{bin}]$ e la tronca a un indice intero; il risultato è una tupla usata come chiave della Q-table (`defaultdict`). Per le velocità (illimitate) si clippa a un intervallo pratico ricavato dalla documentazione Gymnasium.

Preprocessing Atari. Segue lo standard di Mnih et al. [2]: no-op iniziali casuali, frame-skip 4, grayscale, resize 84×84, stack di 4 frame (per dare informazione di moto). I frame sono tenuti in `uint8` nel buffer per risparmiare memoria e normalizzati a $[0, 1]$ solo al momento del training.

3.3 Policy di predizione valutata

MC e TD(0) sono algoritmi di predizione e richiedono una policy *fissa*. La scelta implementativa (documentata anche nel codice) è: addestrare Q-Learning, estrarne la policy greedy $\pi(s) = \arg\max_a Q(s, a)$, e valutare *quella* policy con MC e TD(0). Come confronto si valuta anche una policy casuale.

3.4 Architettura del DQN

La rete è la Nature-CNN [2]: tre layer convoluzionali (8×8/4, 4×4/2, 3×3/1) seguiti da due layer fully connected ($3136 \rightarrow 512 \rightarrow n_{\text{azioni}}$). Per il debug su CartPole esiste anche una MLP a due hidden layer. La rete restituisce $Q(s, \cdot)$ per tutte le azioni in un solo forward pass.

3.5 Training vs evaluation

Per gli algoritmi tabellari la "curva di apprendimento" è il return per episodio *durante* il training (con esplorazione attiva). Per la predizione, invece, si riporta il return degli episodi generati sotto la policy fissa valutata. Per il DQN il training è *step-based* (budget fisso di passi ambiente); la baseline casuale viene valutata separatamente campionando azioni uniformi. I checkpoint dei modelli DQN sono salvati in `results/pong/` per ogni configurazione e seed.

3.6 Riproducibilità

Ogni run fissa il seed con `set_seed` (numpy + torch CPU/CUDA) e `env.reset(seed=...)`. Tutti i risultati riportati sono media su 3 seed $\{42, 123, 456\}$ con banda $\pm 1\sigma$. Gli array grezzi

delle comparazioni di controllo e dell'ablazione Pong sono salvati in `results/` come `.npz`, così che ogni tabella e figura sia rigenerabile senza rieseguire il training.

4 Setup sperimentale

4.1 Hardware e librerie

Gli esperimenti tabellari (CartPole, Acrobot) girano su CPU. Il DQN gira su GPU NVIDIA RTX 5060 (8 GB, architettura Blackwell), che richiede la build PyTorch CUDA 12.8. Librerie: Gymnasium (+ ALE per Atari), PyTorch, NumPy, Matplotlib. La gestione delle dipendenze usa `uv` (`uv sync`); le librerie RL di alto livello (es. Stable-Baselines3) *non* sono usate per il core, come richiesto dalla consegna.

4.2 Iperparametri

Tabella 2: Iperparametri principali. I valori di default seguono le convenzioni di Sutton & Barto; per il DQN sono ridotti rispetto al paper Nature per rientrare nel budget di calcolo (vedi Sez. 6).

	Tabellari (CartPole/Acrobot)	DQN (Pong)
Episodi / step	5000 / 10000 episodi	200 000 step
α (learning rate)	0.1	10^{-4} (Adam)
γ	0.99	0.99
ε	0.1 (costante)	$1.0 \to 0.02$ (lineare, 100k step)
Replay buffer	—	100 000 transizioni
Batch size	—	32
Target update	—	ogni 1000 step
Learning starts	—	10 000 step
Seed	$\{42, 123, 456\}$ (3 run per esperimento)	

4.3 Esperimenti eseguiti

CartPole (`run_cartpole.py`). (1) Q-Learning vs SARSA; (2) sensibilità a $\varepsilon, \alpha, \gamma$ per entrambi gli algoritmi (6 figure); (3) MC vs TD(0) su policy appresa e casuale; (4) casi di fallimento (α elevato, $\varepsilon = 0$).

Acrobot (`run_acrobot.py`). (1) Q-Learning vs SARSA; (2) effetto della risoluzione di discretizzazione (4/6/8/10 bin); (3) sensibilità a ε; (4) MC vs TD(0) su policy appresa.

Pong (`run_pong.py`). Studio di ablazione a 5 configurazioni — DQN completo, senza target network, senza replay, senza entrambi, e Double DQN — più la baseline casuale. Ogni configurazione su 3 seed.

4.4 Criteri di valutazione

Le curve mostrano il return (media su 3 seed, smoothing a media mobile) con banda $\pm 1\sigma$. Le tabelle riportano la performance "finale", definita come media sull'*ultimo 10%* degli episodi di ciascun seed (aggregata poi tra seed): è una stima dell'asintoto più robusta del valore degli ultimissimi episodi, molto rumorosi. Il codice che genera le tabelle (`scripts/make_report_tables.py`) legge solo gli `.npz` salvati.

5 Risultati

Tutti i valori numerici provengono dagli array `.npz` salvati dai runner e sono calcolati da `make_report_tables.py`.

5.1 CartPole — controllo

La Fig. 1 e la Tab. 3 mostrano il confronto Q-Learning vs SARSA *dopo il fix #2* (discretizzazione ristretta ai limiti di terminazione, Sez. 7.5). Entrambi apprendono chiaramente, partendo da ∼30–40, ma il comportamento è cambiato in modo istruttivo rispetto alla versione con bin larghi:

- **Il tetto si è alzato**: le curve superano il vecchio plateau di ∼184 e sono *ancora in salita* a 5000 episodi (picchi ∼250), invece di appiattirsi presto.
- **SARSA ora supera Q-Learning**: return finale 225.2 ± 61.4 contro 173.8 ± 39.0. Nella versione a bin larghi i due erano appaiati a ∼184.
- **Convergenza più lenta e più rumorosa**: con bin più fini ci sono più stati da visitare, quindi servono più episodi e la varianza tra seed cresce (da $\pm 8/\pm 18$ a $\pm 39/\pm 61$).

È un risultato **a doppio taglio**, riportato onestamente: il fix #2 *alza* la performance raggiungibile ma *rallenta* la convergenza — un esempio concreto del compromesso risoluzione/efficienza campionaria della discretizzazione. Q-Learning resta lievemente meno rumoroso di SARSA (varianza minore) ma raggiunge un return più basso in questo budget.

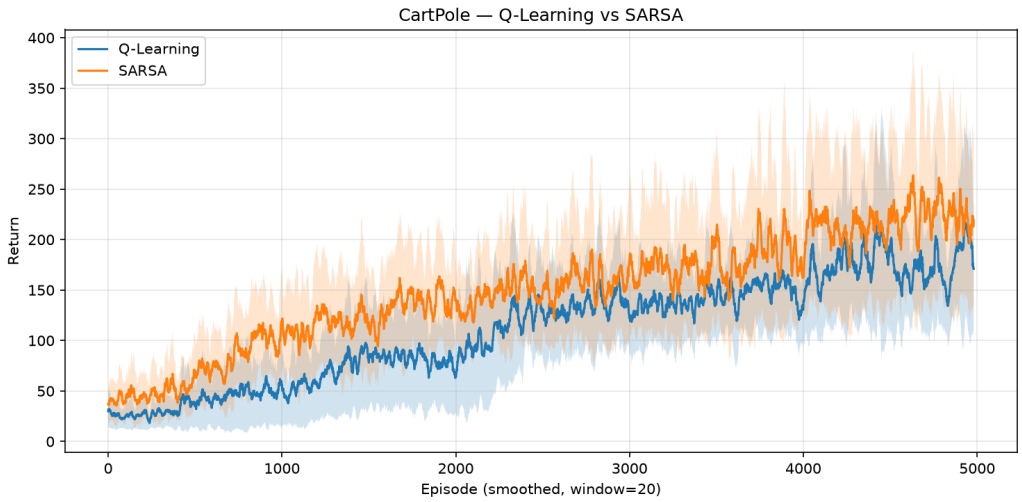


Figura 1: CartPole: Q-Learning vs SARSA dopo il fix #2 (media su 3 seed, banda $\pm 1\sigma$). Le curve sono ancora in salita a 5000 episodi; SARSA (arancio) supera Q-Learning (blu).

Tabella 3: CartPole controllo (discretizzazione fine, fix #2): Q-Learning vs SARSA, media su 3 seed. Il return massimo è 500; con i bin più fini le curve sono ancora in salita a 5000 episodi e SARSA supera Q-Learning.

Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media ± std)	Episodi/seed
Q-Learning	26.6	173.8 ± 39.0	5000
SARSA	40.8	225.2 ± 61.4	5000

5.2 CartPole — sensibilità agli iperparametri

Le figure di sensibilità (una per iperparametro) confermano le attese teoriche:

- α (Fig. 2, sinistra): $\alpha = 0.01$ apprende troppo lentamente ($\sim$30); $\alpha = 0.05$ intermedio ($\sim$110); $\alpha = 0.1$ e $\alpha = 0.5$ raggiungono entrambi $\sim$180–200, con $\alpha = 0.5$ più *veloce* all'inizio. Con la griglia più fine (fix #2) un learning rate più alto aiuta a coprire i molti stati in fretta.
- γ (Fig. 2, destra): valori alti sono migliori ($\gamma = 1.0$ raggiunge $\sim$250 e $\gamma = 0.99$ $\sim$180, mentre $\gamma = 0.9/0.95$ si fermano a $\sim$130). Atteso: in un compito di "sopravvivenza" con reward $+1$/passo conviene valorizzare il lungo termine.
- ε: valori troppo bassi rallentano l'apprendimento, valori troppo alti aggiungono rumore alla policy.

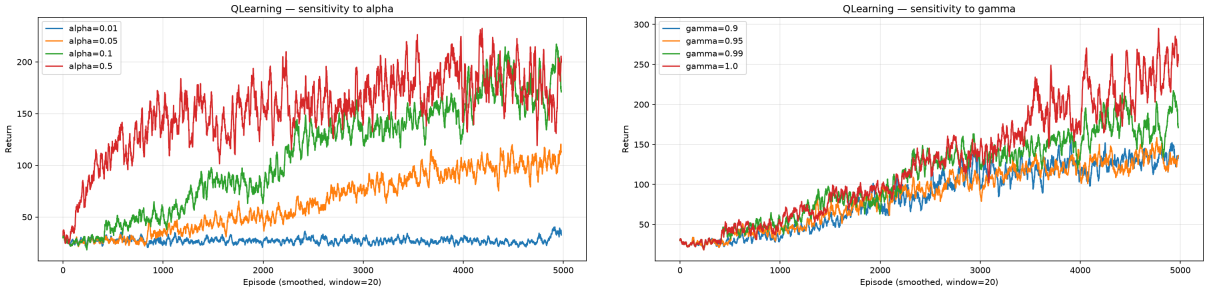


Figura 2: CartPole, Q-Learning: sensibilità ad α (sinistra) e γ (destra). γ alto è la scelta migliore; con la griglia fine anche $\alpha = 0.5$ funziona bene (più veloce di $\alpha = 0.1$).

5.3 CartPole — casi di fallimento

Sono stati riprodotti due fallimenti espliciti (Fig. 3):

- **Learning rate estremo**: con $\alpha = 1.0$ l'agente sostituisce completamente la stima ad ogni passo e resta bloccato a $\sim$65, senza progredire — il fallimento. Da notare che qui i valori *moderati* non falliscono: $\alpha = 0.5$ ($\sim$160) risulta anzi il più veloce, sopra ad $\alpha = 0.1$ ($\sim$140). Il problema è quindi specifico del valore *estremo* $\alpha = 1.0$, non genericamente dell'"α alto".
- **Assenza di esplorazione**: con $\varepsilon = 0$ (e quasi anche con $\varepsilon = 0.01$) l'agente resta intrappolato attorno a $\sim$40, mentre $\varepsilon = 0.1$ raggiunge $\sim$140. È la dimostrazione concreta del dilemma exploration/exploitation.

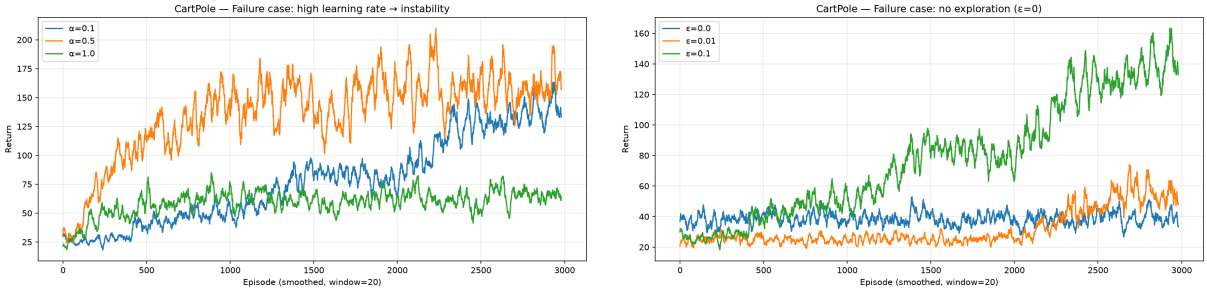


Figura 3: CartPole, casi di fallimento. Sinistra: $\alpha = 1.0$ non progredisce ($\sim$65), mentre $\alpha = 0.5$ è il migliore. Destra: $\varepsilon = 0$ resta bloccato ($\sim$40), $\varepsilon = 0.1$ apprende ($\sim$140).

5.4 CartPole — MC vs TD(0) (predizione)

Il confronto MC vs TD(0) va letto con due grafici distinti. Il *return* degli episodi (Fig. 4) è *identico per costruzione* tra i due metodi: entrambi seguono la stessa policy fissa, quindi generano la stessa

distribuzione di return (media ∼162). Questo grafico non può mostrare la differenza tra i due stimatori — una lezione metodologica in sé (vedi fix #1, Sez. 7.5).

La differenza vive invece nelle *stime* $V(s)$. La Fig. 5 traccia la stima $V(s_{\text{ref}})$ di uno stato di riferimento (lo stato di reset discretizzato, sempre visitato) episodio per episodio: **MC** salta rapidamente al valore corretto (∼90) con alta varianza iniziale (non distorta), mentre **TD(0)** parte da 0 e sale lentamente e in modo liscio (distorta all'inizio per il bootstrap, bassa varianza). È esattamente il compromesso bias/varianza atteso (S&B §6.2).

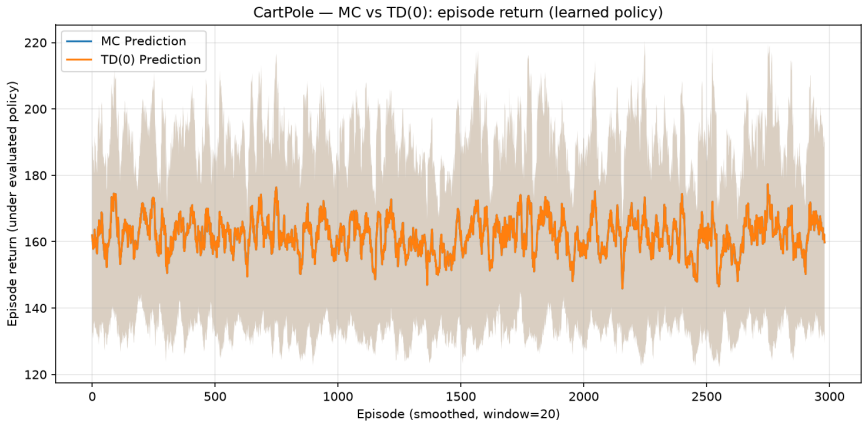


Figura 4: CartPole: MC vs TD(0), *return* della policy appresa. Le curve coincidono perché i due metodi valutano la stessa policy — il return non distingue gli stimatori.

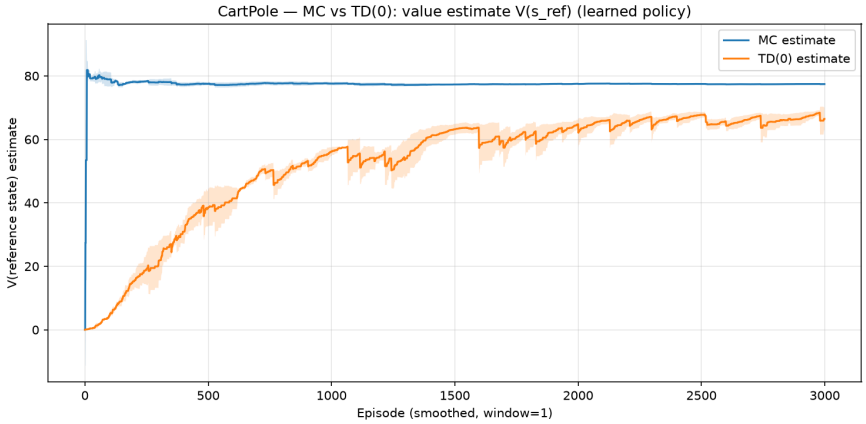


Figura 5: CartPole: MC vs TD(0), *stima* $V(s_{\text{ref}})$ nel tempo (fix #1). MC (blu) salta subito a ∼90 con alta varianza iniziale; TD(0) (arancio) parte da 0 e sale lentamente per il bootstrap: questo è il vero confronto bias/varianza.

5.5 Acrobot — controllo

Su Acrobot (Fig. 6, Tab. 4) entrambi gli algoritmi migliorano da ∼−470 (vicino al caso casuale, −500) fino a un plateau rumoroso attorno a −345/−355. La performance resta ben lontana dall'ottimo (∼−100): l'agente impara a dare qualche spinta utile ma non una strategia di swing-up efficiente. Q-Learning è di nuovo più stabile di SARSA (±10 vs ±31). L'elevata varianza delle curve è coerente con reward sparso e spazio degli stati grande.

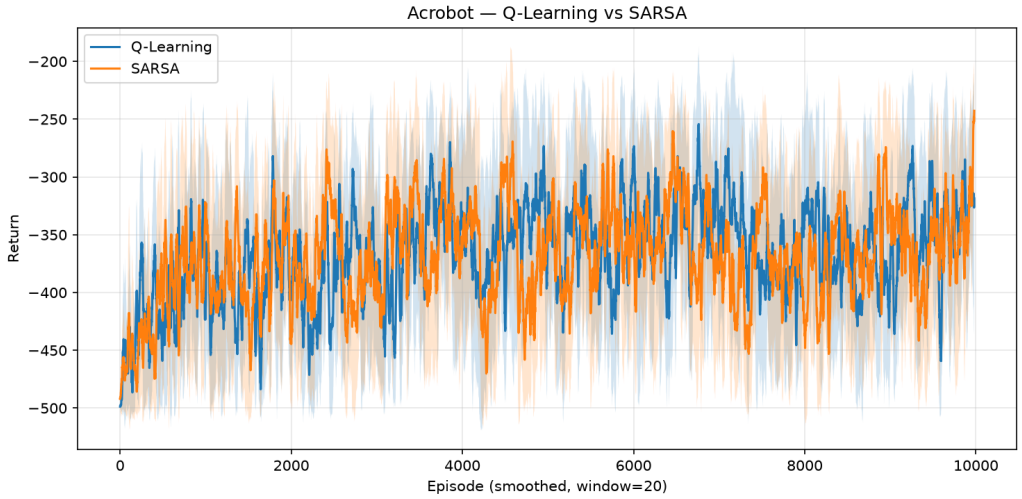


Figura 6: Acrobot: Q-Learning vs SARSA. Miglioramento netto rispetto al casuale (−500) ma lontano dall'ottimo (∼−100); curve molto rumorose.

Tabella 4: Acrobot controllo: Q-Learning vs SARSA, media su 3 seed. Policy casuale ≈ −500; un buon swing-up raggiunge ≈ −100.

Algoritmo	Return iniziale (primi 100 ep)	Return finale (ultimo 10% ep, media ± std)	Episodi/seed
Q-Learning	-468.0	-344.7 ± 10.2	10000
SARSA	-473.2	-354.5 ± 30.9	10000

5.6 Acrobot — risoluzione della discretizzazione

Questo è l'esperimento più istruttivo e con l'esito **più sorprendente** (Fig. 7). L'ipotesi iniziale (annotata anche nel codice) era che aumentare i bin avrebbe *peggiorato* l'apprendimento a causa della sparsità (10 bin $\rightarrow 10^6$ stati). I dati mostrano l'**opposto**: entro 10 000 episodi, **10 bin è la configurazione migliore** (∼−300) e **4 bin la peggiore** (∼−420).

L'interpretazione corretta è che le dinamiche di Acrobot richiedono precisione angolare: con soli 4 bin la discretizzazione è troppo grossolana per rappresentare stati distinti, e ciò domina sull'effetto della sparsità. La "maledizione della dimensionalità" è reale in principio, ma a questo budget di episodi non è ancora il fattore limitante — una lezione metodologica importante: le ipotesi vanno verificate sui dati, non assunte.

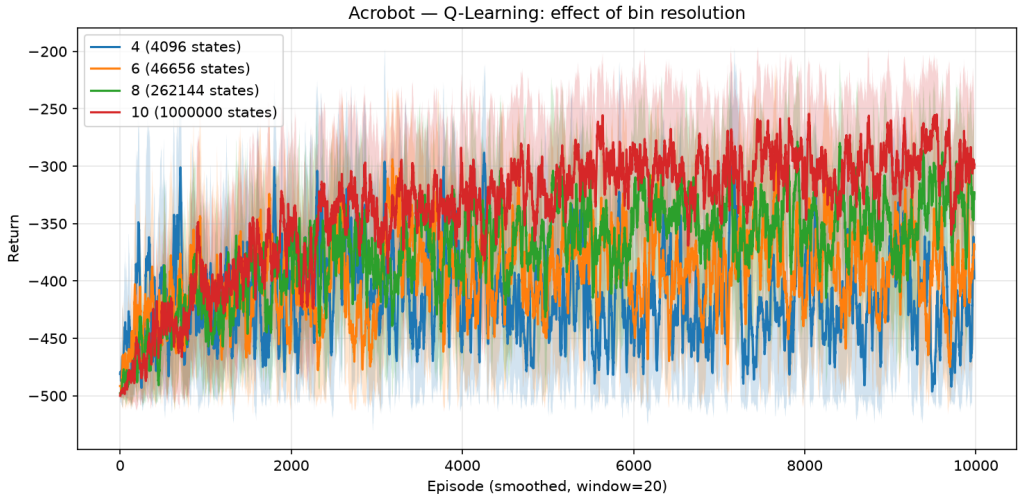


Figura 7: Acrobot: effetto della risoluzione di discretizzazione. Contrariamente all'ipotesi iniziale, 10 bin (rosso) è il migliore e 4 bin (blu) il peggiore entro 10k episodi: la precisione angolare conta più della sparsità a questo budget.

5.7 Acrobot — sensibilità a ε e predizione

Anche la sensibilità a ε (Fig. 8) smentisce l'attesa "reward sparso ⇒ serve più esplorazione": i valori *bassi* ($\varepsilon = 0.05$–0.1) danno i risultati migliori (~ -320), mentre $\varepsilon = 0.4$ è nettamente il peggiore (~ -440). Con ε costante (non decrescente), un valore alto disturba continuamente la policy greedy e impedisce di sfruttare lo swing-up appreso. Il confronto MC vs TD(0) su Acrobot (fix #1, Fig. 9) illustra il **bias di TD in modo ancora più netto** che su CartPole: MC converge quasi subito al valore corretto (~ -92), mentre TD(0) parte da 0 e *scende lentamente*, raggiungendo solo ~ -57 dopo 3000 episodi — ancora lontano dalla convergenza. Lo stato di riferimento (posizione di reset) è $\sim$90 passi dall'obiettivo, e TD propaga il valore un passo alla volta: per uno stato così "lontano" dal reward il bootstrap è lento e la stima resta distorta verso l'inizializzazione (0). MC, usando il return completo, non ha questo problema.

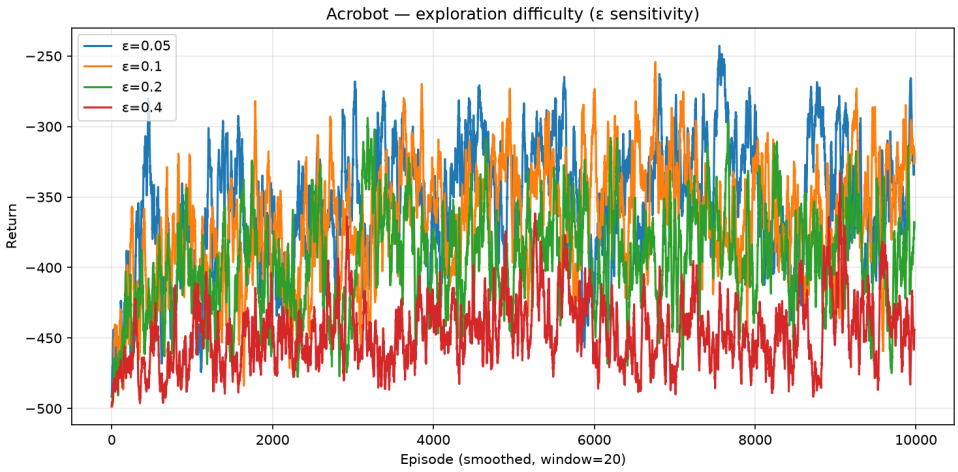


Figura 8: Acrobot: sensibilità a ε. Con ε costante, valori bassi (0.05–0.1) battono valori alti (0.4): troppa esplorazione disturba la policy.

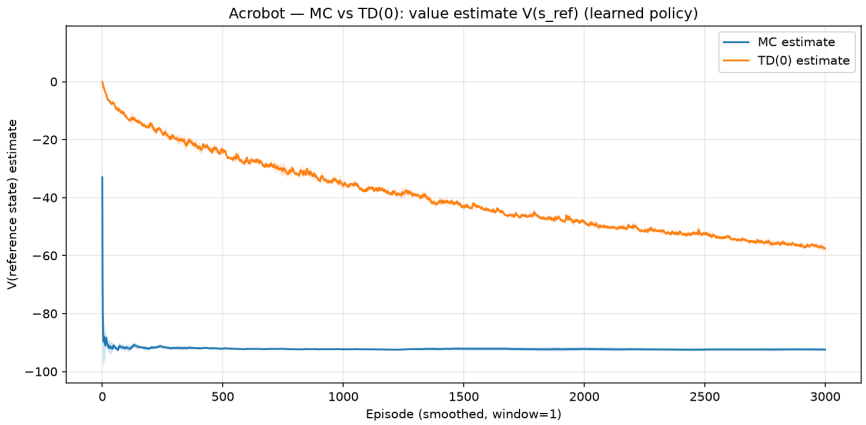


Figura 9: Acrobot: MC vs TD(0), stima $V(s_{\text{ref}})$ nel tempo (fix #1). MC (blu) converge subito a ~ -92; TD(0) (arancio) scende lentamente da 0 e a 3000 episodi è solo a ~ -57: chiara evidenza del bias e della lenta propagazione del bootstrap per uno stato lontano dal reward.

5.8 Pong — studio di ablazione del DQN

Il risultato Pong è il più pulito e "da manuale" del progetto (Fig. 10, Tab. 5). Rispetto alla baseline casuale (≈ -20.4):

- **DQN completo**: miglior risultato, -12.1 ± 0.9. Apprende chiaramente e in modo consistente tra i seed.
- **Double DQN**: -14.2 ± 0.8, apprende con traiettoria simile al DQN completo (in questo regime breve non lo supera).
- **Senza target network**: -16.3 ± 2.8. Apprende ma più lentamente e con varianza molto maggiore (il "target mobile" rende l'addestramento instabile).
- **Senza replay** e **senza entrambi**: -20.6 e -20.5, cioè *nessun apprendimento* — restano al livello casuale.

La conclusione è netta: l'**experience replay è il componente critico** (senza di esso non si apprende affatto), mentre la target network *accelera e stabilizza* ma non è strettamente indispensabile. Si noti anche che il DQN completo produce *meno* episodi per seed ($\sim$134) rispetto alle configurazioni fallite ($\sim$240): apprendere allunga gli scambi, quindi a parità di 200k step si giocano meno episodi — un segnale indiretto di apprendimento.

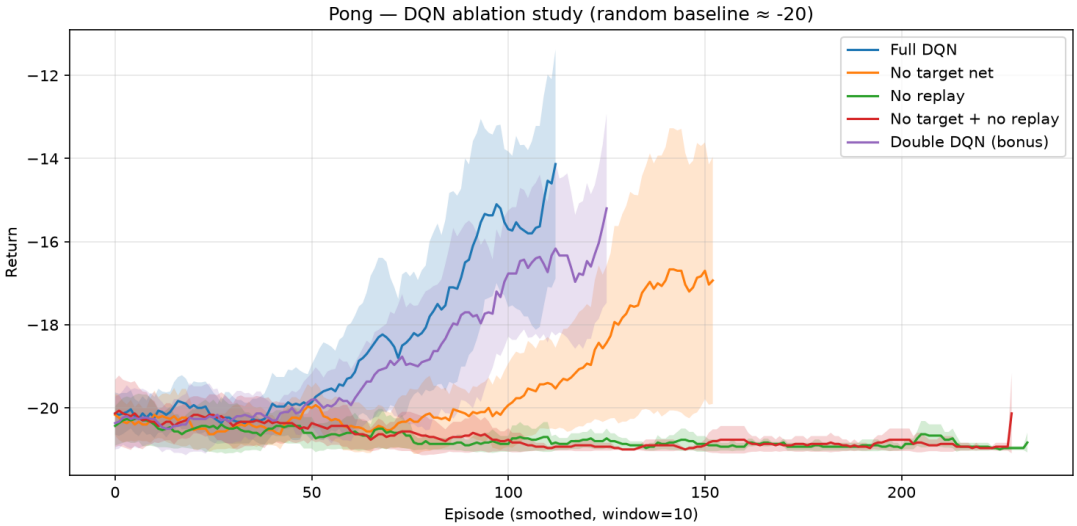


Figura 10: Pong: studio di ablazione DQN (200k step, 3 seed). Replay + target (blu) e Double DQN (viola) apprendono; senza target (arancio) apprende lentamente; senza replay (verde/rosso) resta al livello casuale.

Tabella 5: Ablazione DQN su Pong: performance finale dopo 200k step ambiente, media ± std su 3 seed. Policy casuale ≈ −20.4.

Configurazione	Return finale (ultimo 10% ep, media ± std)	Episodi/seed (media)
DQN completo (replay + target)	-12.1 ± 0.9	134
Double DQN (bonus)	-14.2 ± 0.8	154
Senza target network	-16.3 ± 2.8	187
Senza experience replay	-20.6 ± 0.2	243
Senza target + senza replay	-20.5 ± 0.3	238

6 Analisi critica

6.1 L'agente sta davvero imparando?

Sì, in tutti e tre gli ambienti, ma in misura diversa:

- **CartPole**: apprendimento netto (da ∼30 a ∼175–225, ancora in salita a 5000 episodi dopo il fix #2), riproducibile tra i seed anche se più rumoroso della versione a bin larghi.
- **Acrobot**: apprendimento reale ma parziale (da −470 a −345), lontano dall'ottimo e molto rumoroso.
- **Pong**: apprendimento chiaro sopra il caso casuale per le configurazioni con replay; nessun apprendimento senza replay.

Il fatto che il codice "giri" non basta: la valutazione si basa sul confronto tra performance osservata e attesa teorica, non sull'assenza di errori.

6.2 Convergenza, stabilità e varianza

Nessun ambiente raggiunge una convergenza piatta: le curve tabellari restano rumorose perché ε è costante (l'agente continua a esplorare) e perché α è fisso (media a pesi esponenziali, non convergenza a media esatta). Questo è atteso e coerente con la teoria. In termini di varianza tra

seed Q-Learning è più stabile di SARSA su entrambi gli ambienti (CartPole ±39 vs ±61; Acrobot ±10 vs ±31): l'update off-policy con max risulta qui meno rumoroso della stima on-policy. Su CartPole, però, dopo il fix #2 SARSA raggiunge un return *più alto* (225 vs 174): il vantaggio di stabilità di Q-Learning non si traduce in performance superiore in questo budget di episodi.

6.3 Riproducibilità

Tutti i risultati sono su 3 seed con banda $\pm 1\sigma$; il seeding combinato (numpy + torch + env) rende le run ripetibili. Le bande, spesso ampie (soprattutto Acrobot e predizione), sono riportate onestamente e non nascoste: sono esse stesse un risultato (indicano quanto l'esito dipenda dal seed).

6.4 Training vs evaluation, checkpoint

Per il DQN training ed evaluation sono separati: la baseline casuale è valutata a parte campionando azioni uniformi, e i pesi finali di ogni run sono salvati come checkpoint (`results/pong/<config>/model_seed<`
Un limite onesto: nel loop DQN la "curva" è il return di training (con ε ancora > 0), quindi è una stima leggermente pessimistica della policy greedy; non è stata aggiunta una fase di evaluation puramente greedy separata, che sarebbe un miglioramento naturale.

6.5 Limiti metodologici individuati

Questa sezione riporta i problemi trovati durante la revisione, senza nasconderli.

(1) Confronto MC vs TD(0): da poco informativo a corretto. Nella prima versione i grafici MC vs TD(0) tracciavano il *return della policy valutata*, identico per costruzione tra i due metodi, e non mostravano il compromesso bias/varianza (che riguarda le stime $V(s)$). Questo è stato corretto (fix #1, Sez. 7.5): si traccia ora la stima $V(s_{\text{ref}})$ di uno stato di riferimento episodio per episodio (Fig. 5), che rende visibile la varianza alta di MC contro la traiettoria liscia di TD(0). Resta una misura *locale* (un solo stato); una misura globale (RMSE di V su tutti gli stati, come nel "random walk") richiederebbe un V di riferimento non disponibile in questo setup.

(2) Ipotesi smentite dai dati (Acrobot). Sia l'esperimento sui bin sia quello su ε hanno dato l'esito opposto all'ipotesi iniziale (più bin = meglio; meno esplorazione = meglio). Questo è stato mantenuto e discusso come risultato, non "corretto" per farlo combaciare con l'attesa. La lezione: la curse of dimensionality e la difficoltà di esplorazione sono reali in principio, ma il loro effetto dipende dal budget di episodi e dallo schema di ε (costante vs decrescente).

(3) Asse x in "episodi" nell'ablazione Pong. Le curve Pong sono in funzione dell'*indice di episodio*, ma le configurazioni raggiungono numeri di episodi diversi a parità di 200k step. Confrontarle allo stesso indice di episodio non è un confronto a parità di calcolo esatta; un asse in "step ambiente" sarebbe più corretto. La conclusione qualitativa (replay indispensabile) resta comunque valida.

(4) Persistenza parziale dei dati grezzi. I runner salvano come `.npz` solo le comparazioni di controllo e l'ablazione Pong; sensibilità, fallimenti e predizione producono solo figure. Le tabelle usano quindi solo dati realmente persistiti; le altre analisi sono discusse qualitativamente dalle figure. Salvare anche quegli array sarebbe un miglioramento di riproducibilità.

6.6 Buono per un progetto didattico, non "research-grade"

I risultati sono pienamente adeguati agli obiettivi didattici della consegna: mostrano apprendimento, confronti corretti e fallimenti analizzati. Non sono "research-grade": CartPole non raggiunge 500 (limite della discretizzazione, alzato ma non eliminato dal fix #2), Acrobot resta lontano dall'ottimo, e Pong si ferma a ~ -12 (un DQN addestrato per 50M step, non 200k, arriverebbe a punteggi positivi). Questi divari sono attesi e coerenti con il budget di calcolo scelto, e sono dichiarati esplicitamente.

7 Analisi sperimentale e lezioni apprese

Questa sezione documenta il *processo* sperimentale, non solo i risultati finali: cosa ci si aspettava, cosa ha funzionato, cosa no, quali problemi sono emersi durante lo sviluppo e quali correzioni sono state applicate. L'obiettivo è una discussione onesta, con ogni affermazione ancorata all'evidenza sperimentale (Sez. 5).

7.1 Obiettivi iniziali di ciascun esperimento

- **CartPole controllo**: verificare che i metodi tabellari apprendano davvero e confrontare on-policy (SARSA) vs off-policy (Q-Learning).
- **Sensibilità** $(\varepsilon, \alpha, \gamma)$: mostrare l'effetto degli iperparametri sulla velocità e stabilità dell'apprendimento.
- **MC vs TD(0)**: illustrare il compromesso bias/varianza tra i due stimatori di predizione.
- **Casi di fallimento**: rendere visibili instabilità (α estremo) e mancata esplorazione ($\varepsilon = 0$).
- **Acrobot**: applicare gli stessi metodi a un ambiente più difficile (reward sparso, stato 6-D) e discutere i limiti della discretizzazione.
- **Pong/DQN**: implementare un DQN da zero e isolare, per ablazione, il contributo di experience replay e target network; testare Double DQN.

7.2 Cosa ha funzionato

- **Apprendimento tabellare corretto**: su CartPole entrambi gli algoritmi salgono da $\sim$30 fino a $\sim$175–225 (ancora in salita a 5000 episodi dopo il fix #2); Q-Learning mostra varianza tra seed più bassa (Tab. 3).
- **Sensibilità coerente con la teoria**: $\alpha = 0.1$ è il compromesso migliore, γ alto vince su un compito di sopravvivenza (Sez. 5.2).
- **Casi di fallimento riproducibili**: $\alpha = 1.0$ e $\varepsilon = 0$ producono i fallimenti attesi (Sez. 5.3).
- **Ablazione DQN da manuale**: replay indispensabile, target network acceleratrice; risultato netto e riproducibile (Fig. 10, Tab. 5).
- **Riproducibilità**: media su 3 seed con bande, seeding combinato (numpy+torch+env), array grezzi salvati in `.npz` e tabelle generate da quei dati.
- **Bonus Double DQN**: implementato e funzionante (apprende in modo simile al DQN completo).

7.3 Cosa non ha funzionato (o ha sorpreso)

- **Il grafico MC vs TD(0) iniziale era poco informativo**: mostrava curve identiche, perché tracciava il return della policy condivisa e non le stime $V(s)$. Corretto (fix #1, Sez. 7.5).
- **Due ipotesi su Acrobot smentite dai dati**: bin più fini sono risultati *migliori* (non peggiori) e ε basso batte ε alto (Sez. 5.6, 5.7). Risultati mantenuti e analizzati, non "aggiustati".

- **CartPole non raggiungeva 500** ma si fermava a ∼184: un *tetto di discretizzazione*, non un bug. Il fix #2 lo ha alzato (∼225) ma al prezzo di convergenza più lenta e varianza maggiore — un miglioramento *a doppio taglio*, non netto (vedi Sez. 7.5).
- **Acrobot lontano dall'ottimo** (∼−345 vs ∼−100) e con curve molto rumorose: atteso per reward sparso e spazio grande.
- **Double DQN non supera il DQN base** nel regime breve (200k step): la riduzione del maximization bias non è determinante con così pochi step; comportamento noto in letteratura, non un errore.

7.4 Problemi incontrati durante lo sviluppo

Questi sono problemi *implementativi* reali (distinti dai limiti intrinseci dell'RL discussi sotto):

- **GPU Blackwell (RTX 5060)**: richiede la build PyTorch CUDA 12.8; con build più vecchie la GPU non viene rilevata.
- **Pipeline Atari, tre trappole risolte**: (i) l'ambiente `-v5` ha già un frame-skip interno, che combinato con `AtariPreprocessing` darebbe uno skip di 16 frame ⇒ si forza `frameskip=1` alla creazione; (ii) in Gymnasium 1.x `FrameStack` è stato rinominato `FrameStackObservation`; (iii) i frame impilati arrivano già in formato channel-first `(4,84,84)`, quindi *non* va applicata alcuna trasposizione prima della CNN. Un errore su uno qualsiasi di questi tre punti produce crash o input malformati alla rete.
- **Registrazione ALE**: con `ale-py` recente serve `gym.register_envs(ale_py)`, altrimenti `gym.make("ALE/...")` fallisce.
- **Memoria del replay buffer**: 1M transizioni Atari a frame interi sarebbero ∼28 GB; si è limitato il buffer a 100k transizioni e si conservano i frame in `uint8`, normalizzando solo al training.
- **Backend Matplotlib**: forzato ad `Agg` (non interattivo) per salvare le figure senza GUI.
- **Conflitti di versione nel codice**: durante la gestione con `git` sono rimasti dei marcatori di merge non risolti in alcuni file (Atari/DQN/plotting). Sono stati risolti mantenendo la versione *effettivamente eseguita* (quella che ha prodotto i risultati Pong), e verificati con un test di import dell'intero pacchetto.

7.5 Correzioni applicate

Fix #1 — confronto MC vs TD(0) reso significativo. *Cosa*: oltre al return della policy, si traccia ora la stima $V(s_{\text{ref}})$ di uno stato di riferimento episodio per episodio. *Perché*: il return è identico per costruzione tra i due metodi (stessa policy), quindi non può mostrare la differenza; la differenza bias/varianza vive nelle stime V. *Effetto*: su CartPole la stima MC salta rapidamente al valore corretto (∼90 sulla policy appresa) con alta varianza iniziale, mentre TD(0) parte da 0 e sale lentamente (distorsione da bootstrap) in modo liscio — il quadro classico di Sutton & Barto §6.2 (Fig. 5). *Un bug scoperto e corretto durante questa verifica*: la prima scelta dello stato di riferimento era il "bin centrale" su ogni dimensione. Su Acrobot ciò dà uno stato *fisicamente impossibile* — l'osservazione codifica gli angoli come $[\cos\theta, \sin\theta, \dots]$ e $\cos\theta, \sin\theta$ non possono essere entrambi ≈ 0 — quindi quello stato non viene mai visitato e la stima restava piatta a 0. Corretto usando lo *stato di reset discretizzato*, sempre raggiungibile e visitato ad ogni episodio (Fig. 9). *Risolto?* Sì, entro l'approccio scelto (un singolo stato di riferimento); una misura globale (RMSE di V su tutti gli stati) richiederebbe un V di riferimento non disponibile qui.

Fix #2 — discretizzazione CartPole più efficiente. *Cosa*: gli intervalli di clipping di posizione e angolo sono stati ristretti dai limiti dello spazio di osservazione (± 4.8, ± 0.418) ai limiti di *terminazione* (± 2.4, ± 0.21). *Perché*: oltre i limiti di terminazione l'episodio è già finito, quindi metà dei bin di posizione e angolo era sprecata su stati che non possono verificarsi. *Effetto (a doppio taglio, non netto)*: raddoppia la risoluzione utile sulle due dimensioni più importanti

e *alza il tetto* — le curve superano il vecchio plateau di ∼184 e SARSA arriva a ∼225. Ma più stati significano convergenza *più lenta* (curve ancora in salita a 5000 episodi) e *varianza maggiore* tra seed (±39/±61 contro ±8/±18). Inoltre l'ordine cambia: SARSA ora supera Q-Learning (Tab. 3). *Risolto?* Il "tetto" si alza ma non sparisce: resta un limite di ogni metodo tabellare su stato continuo (serve approssimazione di funzione).

7.6 Limiti residui

Alcuni limiti restano anche dopo le correzioni. È importante distinguere quelli *intrinseci all'RL* da eventuali difetti di implementazione: i seguenti sono caratteristiche attese, non bug.

- **Efficienza campionaria (sample efficiency) bassa**: i metodi tabellari richiedono migliaia di episodi (5k CartPole, 10k Acrobot) per risultati modesti. È inerente al tabular senza generalizzazione.
- **Stabilità di convergenza**: le curve non si appiattiscono mai del tutto. Con ε e α *costanti*, l'agente continua a esplorare e pesa le esperienze recenti in modo esponenziale: la fluttuazione asintotica è attesa, non un errore.
- **Sparsità del reward (Acrobot)**: reward -1/passo fino all'obiettivo rende l'assegnazione del credito difficile; spiega la convergenza lenta e la distanza dall'ottimo.
- **Sensibilità agli iperparametri**: i risultati dipendono marcatamente da $\alpha, \gamma, \varepsilon$ e dal numero di bin (Sez. 5.2, 5.6). Tipico dell'RL.
- **Varianza da seed / casualità**: le bande $\pm 1\sigma$ sono a volte ampie (Acrobot, predizione). Con solo 3 seed la stima dell'incertezza è essa stessa rumorosa; è un compromesso di budget, dichiarato apertamente.
- **Vincoli computazionali (Pong)**: 200k step contro i 50M del paper Nature; per questo Pong si ferma a ~ -12 e non a punteggi positivi. È una scelta di budget, non un difetto dell'implementazione.
- **Strategia di esplorazione**: ε-greedy costante è la più semplice; su Acrobot un valore alto *costante* penalizza (Sez. 5.7). Un annealing sarebbe più adatto.
- **Generalizzazione / discretizzazione**: la discretizzazione uniforme perde precisione (tetto di CartPole; Acrobot lontano dall'ottimo). Non è overfitting in senso classico ma *sotto-rappresentazione* dello stato: il limite è la griglia, non i dati.
- **Predizione, misura locale**: il fix #1 mostra la varianza su *un* stato di riferimento, non l'errore globale su V.

7.7 Possibili miglioramenti futuri

Realistici, oltre gli obiettivi del corso:

- **Annealing di ε e decadimento di α**: curve più pulite e migliore convergenza asintotica.
- **Approssimazione di funzione per Acrobot**: tile coding o una piccola rete al posto della tabella, per superare la maledizione della dimensionalità.
- **DQN più lungo e varianti**: più step, più Dueling DQN e Prioritized Experience Replay, verso punteggi positivi su Pong.
- **Predizione più rigorosa**: RMSE di V rispetto a un riferimento Monte Carlo ad alta campionatura, come nell'esercizio `random_walk`.
- **Ablazione Pong in funzione degli step ambiente**: confronto a parità esatta di calcolo.
- **Fase di valutazione puramente greedy per il DQN**: separare il return di training (con esplorazione) da quello della policy finale.

8 Conformità alla consegna

La Tab. 6 mappa ogni requisito della consegna sul relativo deliverable. Legenda stato: ✓ soddisfatto, ˜ parziale.

Tabella 6: Matrice di conformità requisiti → implementazione.

Requisito	Stato	Evidenza
Env 1 — CartPole (comune)		
Q-Learning, SARSA, MC, TD(0) "from scratch"	✓	`rl_lab/algorithms/tabular/`
Discretizzazione stato continuo	✓	`envs/discretizer.py`; Tab. 1
Curve di apprendimento (reward vs episodi)	✓	Fig. 1
Sensibilità a $\varepsilon, \alpha, \gamma$	✓	Sez. 5.2, 6 figure
SARSA vs Q-Learning (empirico + teorico)	✓	Sez. 5.1, 2
MC vs TD(0) (bias/varianza/stabilità)	✓	Fig. 5 (stima V); vedi fix #1 in Sez. 7.5
≥ 1 caso di fallimento	✓	Sez. 5.3, 2 casi
Env 2 — Acrobot (assegnato)		
Stessi algoritmi applicati	✓	`run_acrobot.py`
Difficoltà di esplorazione (reward sparso)	✓	Sez. 5.7
Necessità di discretizzazione/approssimazione	✓	Sez. 5.6 (esperimento sui bin)
Env 3 — Pong (Atari, assegnato)		
DQN "from scratch" (replay + target)	✓	`algorithms/deep/dqn.py`
Ablazione con/senza target network	✓	Fig. 10, Tab. 5
Ablazione con/senza experience replay	✓	Fig. 10, Tab. 5
Apprendimento vs policy casuale	✓	baseline ≈ -20.4; DQN -12.1
Double DQN (bonus)	✓	flag `use_double_dqn`; Sez. 5.8
Requisiti trasversali		
Media su 3 seed + deviazione standard	✓	tutte le figure/tabelle
Riproducibilità	✓	seeding, `.npz`, `uv.lock`
No librerie RL di alto livello (Env 1/2)	✓	core scritto a mano; SB3 non usato
Report 10–15 pagine, con failure analysis	✓	questo documento (LaTeX)

Nota sul confronto MC vs TD(0). In una prima versione questo era l'unico requisito solo parzialmente soddisfatto: la visualizzazione mostrava il return della policy (identico per costruzione) e non il compromesso bias/varianza. Con il fix #1 (Sez. 7.5) si traccia ora anche la stima $V(s_{\text{ref}})$ episodio per episodio, che espone la varianza alta di MC contro la traiettoria liscia di TD(0): il requisito è ora pienamente soddisfatto. Resta come possibile estensione una misura globale (RMSE di V su tutti gli stati).

9 Origine del codice

Per trasparenza si dichiara la provenienza di ogni componente. Il dettaglio completo è in `docs/origine_codice.md`; qui si riporta la sintesi.

Tabella 7: Origine dei moduli. *Adattato* = derivato dal materiale del corso; *Nuovo* = scritto da zero per questo progetto; *Standard* = replica di architetture note (citate).

Componente	Origine	Fonte / note
Q-Learning, SARSA	Adattato	esercitazioni del corso (`control.py`); adattato all'API Gymnasium e agli stati discretizzati
MC prediction, TD(0)	Adattato	`prediction.py`, esercizio "random walk"
Policy ε-greedy	Adattato	`Esercitazione_RL`
Discretizzatore	Nuovo	gli ambienti del corso erano già discreti
Preprocessing Atari	Adattato	`Tutorial_Gymnasium`; wrapper Gymnasium (non SB3)
Replay buffer	Nuovo	concetto da Mnih et al. [2]
Rete Nature-CNN / MLP	Standard	architettura da [2], implementazione PyTorch originale
Loop di training DQN	Nuovo	parte più originale; il corso forniva solo DQN basato su SB3 (vietato)
Double DQN	Nuovo	da [5], $\sim$5 righe sul DQN base
Runner multi-seed	Ispirato	pattern `run_bias_experiment` del corso
Plotting (bande $\pm\sigma$)	Nuovo (parz. adattato)	`moving_average` dal corso; curve multi-seed nuove
Script per ambiente, tabelle	Nuovo	`scripts/`

In sintesi: i *quattro algoritmi tabellari* sono adattati dalle esercitazioni; il *DQN*, il discretizzatore, l'infrastruttura multi-seed e tutti gli script sono *scritti da zero*. Nessun codice Stable-Baselines3 è usato nel core, come richiesto dalla consegna.

10 Conclusioni

10.1 Risultati raggiunti

Il progetto soddisfa i requisiti della consegna: i quattro algoritmi tabellari e il DQN sono implementati da zero e applicati ai tre ambienti; ogni risultato è mediato su 3 seed con bande di deviazione standard; sono presenti le analisi di sensibilità, i confronti richiesti (SARSA vs Q-Learning, MC vs TD(0)), i casi di fallimento e lo studio di ablazione del DQN con il bonus Double DQN.

10.2 Cosa si è imparato

- La discretizzazione non è neutra: la sua risoluzione governa sia il tetto di performance sia la velocità di convergenza. Restringere i bin di CartPole ai limiti di terminazione (fix #2) alza il tetto (le curve superano 184 e arrivano a $\sim$225) ma rallenta la convergenza e aumenta la varianza. La risoluzione ottima dipende dall'ambiente e dal budget di episodi (Acrobot: 10 bin migliori di 4, contro l'intuizione).
- On-policy (SARSA) vs off-policy (Q-Learning): Q-Learning è più stabile tra i seed, ma su CartPole (fix #2) SARSA raggiunge un return più alto: stabilità e performance non coincidono necessariamente.
- Nel DQN, l'experience replay è il componente decisivo per l'apprendimento; la target network migliora stabilità e velocità.

- Le ipotesi teoriche vanno *verificate* sui dati: due esperimenti Acrobot hanno dato l'esito opposto all'attesa, e questo è stato analizzato invece che nascosto.

10.3 Possibili miglioramenti futuri

- Rendere informativo il confronto MC vs TD(0) tracciando l'errore sulle stime $V(s)$ (o la varianza dei target), non il return della policy.
- Aggiungere una fase di evaluation puramente greedy per il DQN e graficare l'ablazione Pong in funzione degli *step* ambiente.
- Provare uno schedule di ε decrescente per i metodi tabellari, per confrontare esplorazione costante vs annealing.
- Persistere come `.npz` anche i dati di sensibilità/fallimento per una riproducibilità completa delle tabelle.
- Estendere il budget di training del DQN (verso punteggi positivi su Pong), tempo e hardware permettendo.

10.4 Valutazione complessiva

Coerentemente con l'obiettivo didattico della consegna — comprensione e analisi critica più che performance assoluta — il progetto raggiunge i suoi scopi. I limiti individuati sono dichiarati esplicitamente e trasformati in spunti di miglioramento, il che è a sua volta parte dell'analisi richiesta.

Riferimenti bibliografici

[1] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.

[4] Mark Towers et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[5] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.